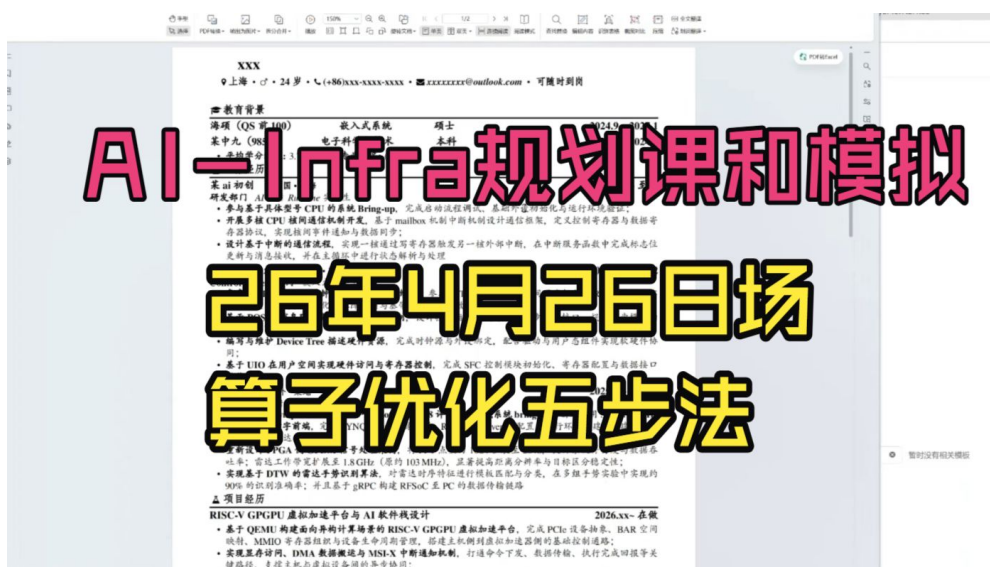


课程笔记

算子优化五步法——AI-Infra 规划课与模拟面

五道口纳什 & Codex

2026-07-18



视频作者/频道：Bilibili：五道口纳什

发布日期：2026-04-26

视频时长：1 小时 12 分

视频链接：<https://www.bilibili.com/video/BV1uaofBREJE/>

目录

1 开场与概览	2
1.1 本章小结	2
2 AI-Infra 岗位图谱：七个角色分类	3
2.1 本章小结	3
3 AI-Infra 技术栈全景	4
3.1 本章小结	5
4 算子优化五步法	5
4.1 第一步：Profiling（性能剖析）	5
4.2 第二步：理论分析（Memory-Bound vs Compute-Bound）	5
4.3 第三步：动手编码（Implementation）	6
4.4 第四步：优化策略实施	6
4.5 第五步：验证与迭代	7
4.6 本章小结	7
5 学员案例诊断：从部署到算子——纵深突破的路径	8
5.1 学员一：从 AI 芯片 Runtime 到算子开发	8
5.2 学员二：从框架调用到底层定制	8
5.3 学员三：把算子项目从「有点虚」做到「踏实」	9
5.4 本章小结	10
6 量化：边缘端与云端的分野	10
6.1 QAT vs PTQ	10
6.2 边缘端部署的特殊性	10
6.3 本章小结	11
7 AI-Infra 职业规划全景	11
7.1 校招时间线	11
7.2 AI 芯片公司 vs 互联网公司	11
7.3 关于华为和安防公司	11
7.4 本章小结	12
8 总结与延伸	12
8.1 讲者结尾总结	12
8.2 核心方法论提炼	12
8.3 开放问题与进一步方向	13
8.4 拓展阅读	13

1 开场与概览

本场是五道口纳什于 2026 年 4 月 26 日举办的 AI-Infra 规划课与模拟面，时长约 73 分钟。课程采用一对多模拟面试的形式，依次为三位背景各异的同学进行简历诊断和职业规划，穿插讲解 AI-Infra 领域最核心的技能与方法论。

三位学员分别代表 AI-Infra 求职者最常见的三种起点：

- 学员一：嵌入式/运行时开发背景，希望从 AI 芯片公司过渡到互联网 AI-Infra
- 学员二：深度学习部署项目经验（Paddle Inference + TensorRT），想突破上层应用天花板
- 学员三：部署流程整合经验，刚开始涉足算子开发（华为昇腾）

虽然场景不同，但讲者给出的诊断和建议高度一致：**纵深不足是共性瓶颈**。停留在框架调用和流程整合层面的项目经验，无法在 AI-Infra 面试中形成差异化。核心突破路径是——向下穿透到框架内部，掌握算子优化全流程。

本课最核心的方法论

算子优化五步法：**Profiling → 理论分析 → 动手编码 → 优化策略实施 → 验证与迭代**。它是贯穿整场模拟面的红线，也是判断一个 AI-Infra 候选人深浅的核心标准。

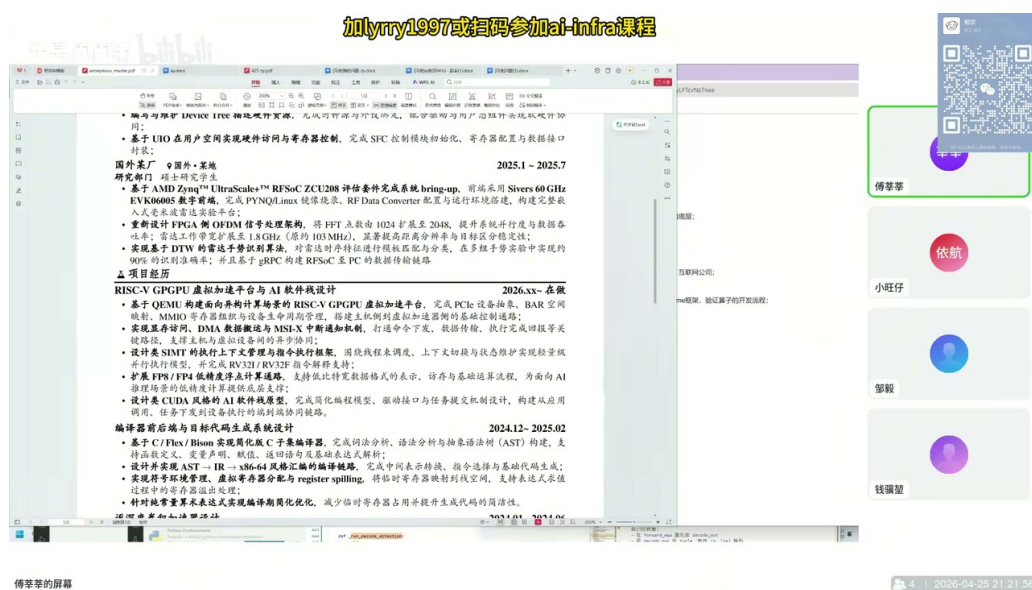


图 1: 开场——讲者为学员一做简历诊断¹

1.1 本章小结

三位学员的问题同出一源：项目停留在部署流水线整合层面，缺乏对推理框架内部机制和算子优化的深度参与。五步优化法是本场课程的方法论锚点，后续各节将依次展开其五个步骤及配套的岗位认知、技术栈、项目策略和职业路径建议。

¹ 视频画面时间区间：00:04:40-00:05:40。

2 AI-Infra 岗位图谱：七个角色分类

讲者将 AI-Infra 相关岗位归纳为七类，并特别指出：并非所有编号都是同一大类，六、七已偏向底层支撑性角色，不是真正的 AI-Infra 工程。

AI-Infra 相关岗位全景
1. 大模型推理加速 / 推理框架开发 (vLLM, SGLang, TensorRT-LLM)
4. 大模型推理系统优化 / 5. 高性能计算 (HPC) 部署
3. 模型训练框架 / 分布式训练 (PyTorch 分布式、DeepSpeed)
2. 算子开发 (CUDA / 国产芯片算子库)
— 以上为 AI-Infra 核心，以下偏支撑 —
6. 云原生 / 容器化推理平台支撑
7. 高性能存储 / KV Cache 传输与分级缓存

图 2: AI-Infra 岗位七分类：1-5 为核心，6-7 偏底层支撑

分类解读

1、3、4、5 其实高度重叠，很多时候面试官自己都划不清边界。真正区分它们的不是岗位编号，而是你实际写的代码在哪一层——是在 PyTorch 层调度算子，还是在 CUDA 层实现算子。6 和 7（云原生部署平台、高性能存储）更多是为推理系统提供基础设施支撑，它们离「AI-Infra 工程师」的核心技能要求有距离。

关键在于理解软硬件分界线在哪里。以 vLLM 为例：

- vLLM 内部既复用了 PyTorch 的一部分计算操作（如两个张量相加直调 torch 自带的），也自己实现了一批自定义算子（如 PagedAttention、FlashAttention）
- 再往下才是 CUDA / cuBLAS / cuDNN，这一层才是真正的软硬件分界线
- 所谓「做底层」，往往指的就是 CUDA/cuBLAS 这一层——在这里写算子，而不是在 PyTorch 层调度

底层 ≠ \$

同样是「底层」，帧级运行时（CPU 中断、裸机调度）和 CUDA 算子开发的底层含义完全不同。当你面对 AI-Infra 面试官时，他们关心的底层是 L1/L2 Cache、向量化寄存器、访存合并——不是 CPU 中断处理。

2.1 本章小结

AI-Infra 岗位的七个分类中，1-5 构成核心圈，6-7 为支撑层。理解岗位的关键不在于编号，而在于理解软硬件分界线在哪一层，以及你在那一层写什么代码。

3 AI-Infra 技术栈全景

讲者以一个清晰的堆栈模型描绘了 AI-Infra 技术路线，自顶向下分为五个层次：

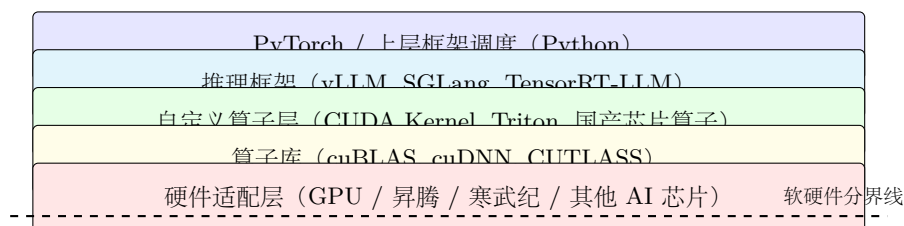


图 3: AI-Infra 技术栈五层模型：箭头为软硬件分界线

每层的语言生态：

- **PyTorch 层**：Python 主导。面向模型开发和推理调度
- **推理框架层**：Python + C++。框架本身多用 Python，但内部热点路径通常用 C++ 自定义算子加速
- **CUDA 算子层**：C++（NVCC 编译）。NVCC 本质是 C++ 编译器，kernel 中可以直接使用 C++ 语法
- **算子库层**：C++。cuBLAS、CUTLASS 均以 C++/模板元编程实现
- **硬件适配层**：C/汇编。不同厂商芯片的驱动与运行时

C++ 还是 Python?

AI-Infra 工程师不需要二选一。PyTorch 是最上层的接口，基本用 Python；CUDA Kernel 是真正的底层性能优化，必须用 C++。实际工作中需要掌握两者的编程方式，尤其在调试时需要同时追踪 Python 侧的调用链和 C++ Kernel 的执行路径。PyTorch 通常花一两周系统过一下即可，不用精读源码；但 CUDA 需要深入掌握，因为算子优化的所有手段都与硬件强绑定。

讲者推荐的学习顺序：

1. 先用 PyTorch 把《动手学深度学习》的例子跑一遍，建立模型侧直觉
2. 接触第一个 CUDA 程序（如简单的向量加），理解 kernel launch 语法
3. 在 CUDA 层面学习核心优化技术：共享内存、寄存器使用、bank conflict 规避
4. 回到上层看推理框架（如 vLLM），理解它是怎么调用下层算子的

计算机体系结构比操作系统更重要

对于算子优化而言，机组（计算机组成原理）的知识远重于操作系统。L1/L2 Cache 的命中策略、向量化寄存器的使用、内存分级的带宽特性——这些在日常的算子加速编码中是高频思考对象，而操作系统的调度与中断机制在算子层面反而感知不强。

3.1 本章小结

AI-Infra 技术栈遵循「上 Python 下 C++」的格局。PyTorch 是顺手掌握的工具，CUDA/C++ 才是核心竞争力。学习路径自上而下建立模型直觉，自下而上掌握性能优化——缺一不可。

4 算子优化五步法

这是整场课程的灵魂。讲者在为三位学员做项目诊断时，反复回到这五个步骤，因为它们构成了判断一个算子项目是否「做实」的黄金标准。

4.1 第一步：Profiling（性能剖析）

找到真正的瓶颈，而不是凭感觉猜测。

在写优化代码之前，必须先用 profiling 工具定位热点——是哪个 kernel 消耗了最多的时间？是计算还是访存占主导？

- 使用 nvprof/Nsight Systems 做 GPU profiling
- 关注 kernel 执行时间、内存带宽利用率、计算利用率（occupancy）
- 从 profiler 数据中提取关键指标：FLOPs、DRAM 读写量、算术强度

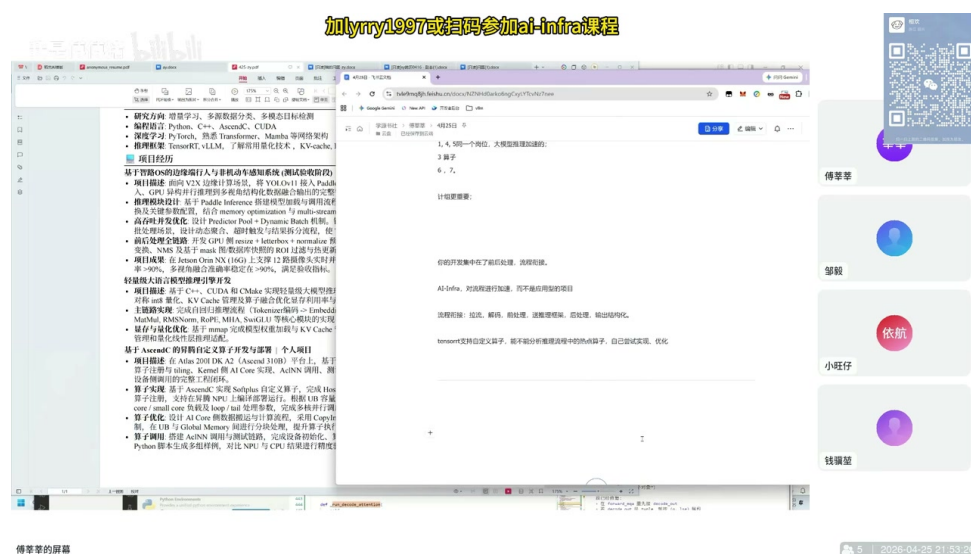


图 4: 讲者在讲解 profiling 的重要性²

4.2 第二步：理论分析（Memory-Bound vs Compute-Bound）

定性判断算子的瓶颈类型，决定优化方向。

理论分析的核心是计算**算术强度**（Arithmetic Intensity）：

$$\text{算术强度} = \frac{\text{总浮点操作数}}{\text{总访存字节数}}$$

将其与硬件峰值对比：

²视频画面时间区间：00:36:30–00:37:30。

- 算术强度低于硬件平衡点 → **Memory-Bound**，优化方向是减少访存、提高缓存命中率
- 算术强度高于硬件平衡点 → **Compute-Bound**，优化方向是提高计算吞吐、利用 Tensor Core

理论分析是区分水平的分水岭

许多候选人能写 CUDA kernel，但讲不出为什么这样写就快——因为没做过理论分析。没有理论分析，优化就是在黑箱中试错。面试官真正在意的不是「你用了哪种优化技巧」，而是「你为什么选择这个技巧」——后者只能用理论分析回答。

4.3 第三步：动手编码 (Implementation)

把理论分析转化为具体的 *kernel* 实现。

编码不是最难的步骤，但有几个关键点：

- 先实现正确性，再追求性能——用 PyTorch 的 CPU 版做 ground truth 对比
- 解耦线程块划分 (blockDim, gridDim) 和核心算法逻辑，方便后续调参
- 保留 benchmark 代码，以便在每一步优化后测量实际加速比

4.4 第四步：优化策略实施

从现象到优化策略的系统转化。

这是体现工程深度的核心步骤。讲者列出常见优化策略的分类：

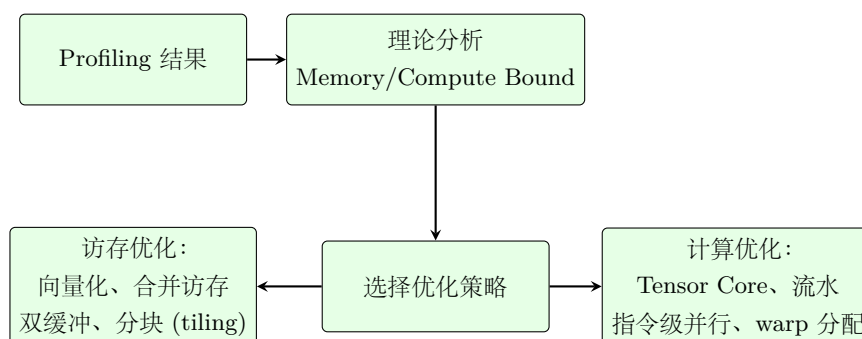


图 5: 从 profiling 结果到优化策略的决策流程

具体策略清单：

- **向量化**：利用 SIMD/SIMT 指令，一次处理多个数据元素
- **合并访存 (Coalesced Memory Access)**：让同一 warp 内的线程访问连续地址，减少内存事务数
- **共享内存分块 (Tiling)**：将数据分片加载到 shared memory，减少全局内存访问
- **双缓冲 (Double Buffering)**：用两个 buffer 轮流加载和计算，隐藏访存延迟
- **流水线 (Pipelining)**：将计算和访存重叠，提高整体吞吐
- **寄存器优化**：控制每线程的寄存器使用量，提高 occupancy

不要跳过理论分析直接调参

常见误区：拿到 profiler 结果后，直接尝试各种优化 trick（加大 shared memory、调 block size），而不先判断 kernel 是 memory-bound 还是 compute-bound。结果可能是花了两天优化的方向根本不对，还引入了新的 bank conflict。必须先定性，再定量。

4.5 第五步：验证与迭代

优化不是一次性的，而是循环迭代的过程。

- 每次优化后重新 profiling，确认性能变化方向和幅度
- 对比理论峰值（如 Roofline Model）与实际值，计算利用率百分比
- 记录每次优化的策略、效果和推理链：“从 XXX 现象 → 选择 YYY 策略 → 性能提升 ZZZ%”
- 这是面试时最能体现工程思维的部分——不是告诉你优化了多少，而是告诉你为什么这样优化

面试中的五步法表达

在简历或面试中描述算子优化项目时，应该天然地按五步法组织：**发现问题 (Profiling) → 定位原因 (理论分析) → 设计方案 (策略选择) → 实现验证 → 性能对比**。只写「用 vectorization 优化了 GEMM，性能达到 cuBLAS 的 92%」是不够的——面试官想知道的是，你是经历了哪些推理步骤才决定用 vectorization 的。

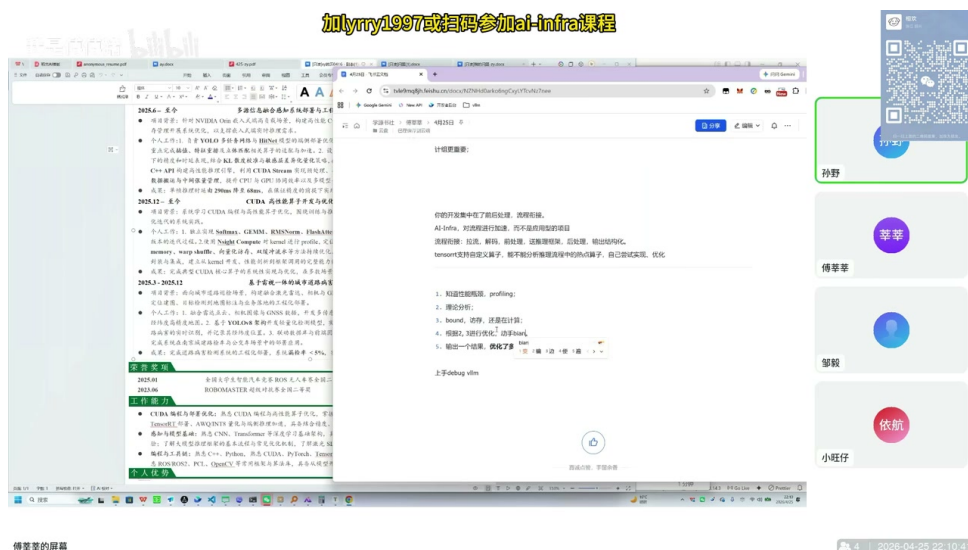


图 6: 讲者在详细阐述五步法的操作细节³

4.6 本章小结

算子优化五步法是一个闭环：Profiling 定位瓶颈、理论分析判断边界、编码实现、策略优化、验证迭代。缺失任何一步都会让项目停留在「会写但说不清」的浅层水平。这五步不仅是技术方法论，也是在面试中展示工程思维的最佳叙事框架。

³视频画面时间区间：00:53:42–00:54:15。

5 学员案例诊断：从部署到算子——纵深突破的路径

本节通过三位学员的具体案例，展示如何将一个「上层应用部署」项目转化为有深度的 AI-Infra 项目。

5.1 学员一：从 AI 芯片 Runtime 到算子开发

背景：在 AI 芯片公司做底层运行时开发（涉及 CPU 启动、中断处理、运行时接口），正在自学 KIM 模拟器搭建 GPU 仿真系统。想秋招冲 AI 芯片公司的算子开发岗，春招再转互联网 AI-Infra。

诊断：

- 当前经历虽偏底层，但与 AI 关联度低——调用 `prefuse/decode` 接口不等于理解推理加速
- KIM 模拟器项目是最大亮点，但还停留在「搭建系统」的初级阶段

方案：

1. 在模拟器上完成后，**在上面实现算子库**——GEMM、Conv、LayerNorm、Attention 等大模型常见算子
2. 特别利用模拟器的 FP8 支持和 SIMT 调度特性——这些在真实 GPU 上很难调试，但模拟器可以精确控制
3. 对每个算子按五步法做 profiling、理论分析、优化、验证——形成完整闭环
4. 封装成算子库，加入 benchmark 对比

项目「差异化」的本质

同样是从零写一个算子——单纯实现一个 CUDA kernel 只能算课后作业；但如果你还能讲清楚它的算术强度、为什么用了分块策略、每层 Cache 命中率的变化、与 cuBLAS 的差距在哪一步——这就是不同量级的项目了。差异不在「做了什么」，而在「想清楚了什么」。

时间线规划：

- 秋招（9-11 月）：投 AI 芯片公司的算子开发岗，用模拟器 + 算子库项目作为核心展示，拿保底 offer
- 之后开始刷 AI-Infra 面试，补充推理框架知识（用 vLLM debug 环境上手）
- 春招（次年 3-4 月）：主投互联网公司的 AI-Infra 岗位，此时已有一套可以从头讲到尾的算子项目

5.2 学员二：从框架调用到底层定制

背景：在某平台基于 Paddle Inference + TensorRT 做了完整的部署项目（拉流 → 前处理 → 推理 → 后处理 → 结构化输出），场景为交通/安防。项目整体流程完整，TensorRT 模型推理正常。

诊断：致命的浅层问题——所有工作都集中在前/后处理和流程衔接上。

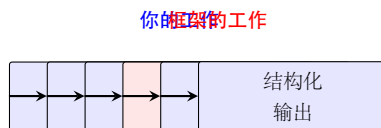


图 7: 典型部署项目的「浅层」特征：工作集中在推理框的输入/输出两端，而非推理本身

TensorRT 的快是框架的能力，不是你的能力。面试官看到这样的项目，会觉得你只是一个 API caller。

方案：

1. **深入 TensorRT 内部**：利用 TensorRT 的插件系统 (IPluginV2)，找到模型中的热点算子，用自己的 CUDA 实现替换
2. **算子融合**：尝试将前处理的一部分计算融合进模型推理图（如归一化、颜色空间转换），减少 CPU-GPU 拷贝
3. **对热点算子做完整的五步法**：profiling → 理论分析 → 编码 → 优化 → 验证
4. 把修改框架内部逻辑的部分单独拿出来，在简历中作为独立亮点突出

安防公司慎入

讲者反复强调：安防公司（海康、大华等）的部署项目存在严重的同质化问题。你做完一个安防项目后，下一个还是安防，无非场景从交通换成校园。这类公司毛利率低、技术栈浅（更多是解决方案集成而非核心研发），对个人技术积累帮助有限。同样学历背景去自驾车企或互联网 AI-Infra 部门，天花板和成长空间差距巨大。

5.3 学员三：把算子项目从「有点虚」做到「踏实」

背景：和学员二的部署项目类似（拉流 → 前处理 → 推理 → 后处理），额外在华为昇腾平台上写过一些算子。转行时间较晚（3 月才开始），目前已过华为实习二面。

诊断：

- 简历上的算子项目写得太笼统——「在大多数情况下能达到高性能库的 82%-92%」，但没有标注具体哪个算子、在哪种输入尺寸下达到这个数字
- 项目的优化流程缺失 profiling 和理论分析两个关键步骤，直接跳到编码优化——这是最典型的「做虚了」的信号
- 向量化、缓存、双缓冲、流水线这些优化技巧都提到了，但缺少从现象推导策略的逻辑链

方案：

1. 为每个算子补充 profiling 数据 (nvprof/msprof)、理论分析（算术强度计算）
2. 按硬件平台区分：安培架构 vs Hopper 架构、英伟达 vs 华为昇腾——不同平台的优化策略差异很大
3. 补全从「现象」到「策略」的推理链：「profiling 发现 L2 cache 命中率仅 30% → 分析输入尺寸发现块大小不匹配 → 调整 tiling 策略 → L2 命中率提升至 78% → 性能提升 1.4x」

4. 面试时能对着自己的项目，从 profiling 数据讲到最终加速比，中间每一步都有理论支撑

简历表达——从笼统到精确

差：「优化了多个算子，多数达到 cuBLAS 的 82%-92%」

好：「对 GEMM ($M=4096, N=4096, K=4096$) 进行优化：profiling 发现 memory-bound (算术强度 38 FLOPS/byte, 低于 A100 平衡点 56) → 采用分块策略 (block 128×128) + 双缓冲 → L2 命中率从 41% 提升至 82% → 最终性能达到 cuBLAS 的 91.3%」

5.4 本章小结

三位学员的问题根源相同：项目停留在上层，缺少纵深。突破路径是向下穿透一层——从框架调用者变成框架修改者、从算子使用者变成算子开发者，并按照五步法把每一个优化决策讲清楚。简历上的项目要精确到具体数据、具体策略和具体推理链，而不是泛泛地列优化技巧。

6 量化：边缘端与云端的分野

6.1 QAT vs PTQ

量化是部署工程师高频接触的话题，但边缘端（CNN 为主）和云端（大模型为主）的量化策略差异很大：

- **QAT (Quantization-Aware Training)**：训练时就引入量化模拟，模型在训练过程中适应低精度表示。适合 CNN 和边缘端——训练时间短，可以承受额外一轮 QAT finetune
- **PTQ (Post-Training Quantization)**：训练完成后做量化，不需要额外训练。大模型的主流方案——没法对 70B 参数量的模型做 QAT

量化在大模型推理中的定位

讲者特别强调：量化只是大模型推理框架中的一个子模块，不是独立赛道。**大模型推理是「爹」，量化是「儿子」**。面试准备的时间分配应该先主后次——理解大模型推理的核心机制（pre-fill/decode 分离、KV Cache 管理、continuous batching），再深入量化、投机采样等补充技术。

6.2 边缘端部署的特殊性

- 算力和内存都非常受限，INT8/FP16 量化几乎必做
- 对延迟极度敏感，batch size=1 场景下需要针对单 batch 做特殊优化
- 需要掌握 TensorRT 的 INT8 校准流程：准备校准数据集 → 收集 activation 分布 → 生成校准表 → 构建 INT8 engine

量化项目如何「做深」

不只是「用 TensorRT 把模型转成 INT8」——如果能在简历中说明：原始模型在具体硬件上的各层精度敏感度、哪些层用了 per-channel 量化哪些用了 per-tensor、校准数据集大小对精度影响的消融实验、最终 INT8 精度损失控制在多少以内——这就是一个有量可循的深度项目。

6.3 本章小结

边缘端仍需要 QAT+PTQ 双掌握（尤其 CNN 场景），但大模型场景下 QAT 已基本退场，PTQ 成为主流。无论在哪个方向，量化的深度体现在对精度-速度 trade-off 的定量分析上，而非仅仅调用 API。

7 AI-Infra 职业规划全景

7.1 校招时间线

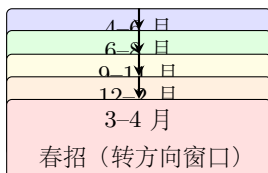


图 8: AI-Infra 方向校招典型时间线

- 秋招先拿保底（AI 芯片公司的算子岗相对好拿），不建议全押 AI-Infra——这类岗位在秋招时的 HC 不多
- 秋招后利用 12 月-2 月集中补齐本领：搭 vLLM debug 环境、写算子项目、刷题
- 春招是 AI-Infra 方向的好窗口——年后 HC 释放，且候选人经过秋招的锤炼后整体水平更低

7.2 AI 芯片公司 vs 互联网公司

- **AI 芯片公司**：算子开发为主，对芯片架构理解要求高。社招转互联网困难——因为互联网公司更看重你对上层框架（vLLM、PyTorch 分布式）的理解
- **互联网公司**：推理框架开发为主，需要理解完整推理 stack。对分布式、KV Cache 管理、continuous batching 等机制要求高
- **关键判断**：你之后想去哪？AI 芯片公司做三年算子开发，再往互联网跳，简历很难被认可——两边技术站的重合度比你以为的少

不要在社招时后悔校招的选择

校招是换方向成本最低的窗口——你面对的是「应届生」这个概念，而不是「有 X 年经验但方向不对口的工程师」。一旦以 AI 芯片公司外包/算子岗身份社招去面互联网 AI-Infra，你会发现面试官问的问题你大多没接触过。能趁早转尽量趁早转。

7.3 关于华为和安防公司

讲者对两个方向给出了明确的风险提示：

华为昇腾：

- 华为技术栈（CANN/昇腾）与行业主流（CUDA/英伟达）的生态差异很大，长期待可能导致技术栈狭窄
- 昇腾的算子开发经验很难平移到互联网的 CUDA 推理框架岗位
- 如果目标是自驾/具身智能——华为实习的匹配度不高；如果目标是国产芯片赛道——华为是优质选择

安防公司（海康/大华等）：

- 本质是制造业，毛利率低导致薪资天花板明显
- 项目以政府解决方案为主，技术积累分散在定制化需求中，个人沉淀较浅
- 同城市同专业背景下，去自驾车企或互联网 AI-Infra 部门优于安防公司

7.4 本章小结

AI-Infra 职业路径的核心建议：秋招保底、春招冲刺、校招窗口期最大程度往目标方向靠。AI 芯片与互联网两条路径之间的技术壁垒比表面看起来大得多，尽早明确方向。华为和安防公司各有风险，需根据个人目标谨慎选择。

8 总结与延伸

8.1 讲者结尾总结

讲者在课程尾声对三位学员的核心建议高度一致：

- 不要全身心等某一家公司（尤其是流程慢的华为）——多线程投递，实习 offer 不是不能走
- 多面试，把面试本身当作学习过程。面试官的反馈有时比自己闭门造车更能暴露短板
- 项目要「做实」：从空洞的上层整合穿透到有 profiler 数据的算子开发，从「会用 TensorRT」到「能改 TensorRT 内部逻辑」
- 车企自驾方向对 AI-Infra 的前景优于安防——无论是薪资、技术含量还是职业延展性

8.2 核心方法论提炼

本场课程的核心知识体系可以压缩为以下结构：

岗位认知层：

- AI-Infra 不是单一岗位，而是分布在 PyTorch → 推理框架 → CUDA Kernel → 算子库 → 硬件五层上的多个角色
- 区分岗位的关键不是编号，而是你在哪一层写代码

能力构建层：

- 五步法是检验算子能力的试金石——少一步就是「做虚了」
- 理论分析（Memory-Bound vs Compute-Bound）是区分「会写」和「懂写」的分界线
- 计算机体系结构知识（Cache、SIMD、内存分级）对算子优化的贡献远超操作系统

职业策略层：

- 校招是转方向成本最低的窗口，尽量在此时入对行
- 安防公司（低天花板 + 浅技术）应尽量避免，即使意味着要等下一个更好的机会
- 面试不是考试——是暴露盲区、获取反馈、逐步校准方向的过程

8.3 开放问题与进一步方向

课程结束时留下的待探索方向：

- 推理框架内部机制的系统学习——vLLM 的 continuous batching、PagedAttention、KV Cache 管理如何对应五步法中的 profiling 和理论分析？
- 跨芯片平台的算子可移植性——CUDA Kernel 能否借助 Triton/MLIR 等中间表示实现一次编写多平台运行？
- 大模型时代的算子分工——FlashAttention 等「超级算子」的出现，是否意味着传统的细粒度算子优化正在被整合性优化取代？

8.4 拓展阅读

- NVIDIA NSight Compute 官方文档：GPU Kernel Profiling 标准工具，对应五步法中的第一步
- CUDA C++ Programming Guide, Chapter 5 (Performance Guidelines): Memory Coalescing、Shared Memory、Occupancy 等核心概念的系统说明
- 「How to Optimize a CUDA Matmul Kernel」系列教程 (Simon Boehm)：从 naive GEMM 到接近 cuBLAS 性能的完整优化旅程
- Roofline Model 论文：Williams et al., "Roofline: An Insightful Visual Performance Model for Multicore Architectures", 2009